PALMGUARD AI • SAWIT DETECTOR

Manual Book & Dokumentasi Kode Penting

Panduan lengkap penggunaan aplikasi deteksi penyakit pelepah kelapa sawit berbasis analisis citra visual dan sistem pakar Forward Chaining, disertai catatan kode sumber inti serta alur build GitHub Actions.

Nama Proyek	PalmGuard AI (`sawit_detector`)
Framework	Flutter (Dart SDK `^3.11.5`)
Dependensi Inti	`camera`, `image_picker`, `cupertino_icons`
Metode Utama	Validasi Citra Pelepah Sawit (`dart:ui` RGBA/HSV/ExG) & Forward Chaining
Repository GitHub	`https://github.com/Galaska77/sawit_detector`

Daftar Isi

1. **Bagian I — Manual Book Pengguna**: Instalasi APK, Menu Utama, Scan Kamera & Galeri Pelepah Sawit, Kuisioner Manual, Pembacaan Hasil, serta Basis Pengetahuan Penyakit.
2. **Bagian II — Dokumentasi Kodingan Penting**: Struktur Folder, Engine Forward Chaining, Validator & Penganalisis Citra Pelepah Sawit, Alur Kamera/Galeri, Penanganan 3 Kondisi Hasil, dan Pipeline CI/CD GitHub Actions.

Bagian I — Manual Book Pengguna

1.1 Persiapan & Instalasi Aplikasi (Android APK)

1. Unduh artefak `sawit-detector-release-apk` dari tab **Actions** pada repository GitHub `Galaska77/sawit_detector`, lalu ekstrak file `.zip` untuk mendapatkan `app-release.apk`.
2. Pindahkan `app-release.apk` ke perangkat Android, lalu ketuk untuk memasang (aktifkan izin *Install unknown apps* bila diminta).
3. Saat pertama kali membuka fitur pemindaian, berikan izin akses **Kamera** dan akses **Galeri / Foto** agar aplikasi dapat memotret maupun membaca gambar pelepah sawit.

1.2 Halaman Beranda (`HomePage`)

Halaman awal aplikasi menampilkan ringkasan sistem pakar dan dua jalur utama deteksi penyakit:

- **Scan Kamera Instan (AI)**: Membuka halaman pemindai kamera langsung sekaligus tombol pemilihan gambar dari galeri.
- **Diagnosis Manual (Kuisioner)**: Membuka konsultasi interaktif berisi 13 pertanyaan gejala lapangan (`G01` – `G13`).
- **Statistik Kebun & Sistem**: Menampilkan jumlah aturan aktif (**6 Aturan**) dan data gejala (**13 Gejala**).
- **Katalog Penyakit Sawit**: Daftar 6 penyakit kelapa sawit (`P01` – `P06`) yang dapat diketuk untuk melihat deskripsi dan saran penanganannya.

1.3 Scan Pelepah Sawit (Kamera & Galeri — `ScanPage`)

Fitur ini dirancang **khusus untuk memindai pelepah atau helai daun kelapa sawit**.

1. **Mode Kamera Langsung**: Arahkan kamera ke pelepah/helai daun sawit pada jarak ±30–60 cm hingga objek berada di tengah kotak bingkai hijau. Gunakan ikon lampu kilat di pojok kanan atas jika area kebun kurang cahaya, lalu tekan tombol **Ambil Foto**.
2. **Mode Galeri (Gambar yang Sudah Ada)**: Tekan tombol **Galeri** di kiri bawah, lalu pilih foto pelepah sawit yang tersimpan di HP.
3. **Validasi Otomatis Objek**: Sebelum mendiagnosis penyakit, sistem memeriksa apakah gambar benar-benar merupakan pelepah/daun sawit. Jika foto berupa wajah manusia, tembok polos, benda ruangan, atau gambar terlalu gelap/silau, sistem akan menolaknya sebagai **Objek Bukan Pelepah Sawit**.

> **Tips agar gambar pelepah terbaca akurat:** pastikan helai daun sawit mengisi area tengah bingkai, gunakan pencahayaan alami yang merata (tidak gelap dan tidak memantul silau), serta hindari memotret wajah atau latar belakang polos secara dominan.

1.4 Diagnosis Manual / Kuisioner (`DiagnosisPage`)

- Aplikasi menampilkan pertanyaan gejala satu per satu dari `G01` hingga `G13` beserta progress bar persentase.
- Pilih **YA** jika gejala terlihat pada pohon sawit di lapangan, atau **TIDAK** jika tidak ditemukan.
- Tekan **Kembali** untuk meninjau atau mengubah jawaban sebelumnya (indikator *Jawaban saat ini: YA / TIDAK* akan tampil).
- Tekan **Selesai Lebih Cepat** kapan saja jika gejala yang relevan sudah selesai diisi.

1.5 Memahami Halaman Hasil (`ResultPage`)

Kondisi Hasil	Tampilan & Indikator	Tindakan Lanjutan
1. Objek Bukan Pelepah Sawit	Kartu peringatan kuning *OBJEK BUKAN PELEPAH SAWIT*, alasan penolakan (mis. terdeteksi kulit/wajah, latar polos, terlalu gelap), dan metrik *Hijau Daun*, *Non-Sawit/Kulit*, *Kecerahan*. Tidak menampilkan penyakit sawit.	Tekan **Scan Ulang Pelepah Sawit** dan arahkan kamera/pilih foto pelepah sawit yang sesuai panduan.
2. Tanaman Sawit Sehat	Kartu hijau *Tanaman Sawit Sehat* disertai persentase *Hijau Sehat*, *Menguning*, dan *Bercak/Busuk* serta rekomendasi pemeliharaan NPK & pengendalian gulma.	Lakukan pemantauan rutin atau tekan **Scan Pelepah Lain** untuk mengecek pohon lain.
3. Indikasi Penyakit Terdeteksi	Kartu diagnosis utama (*TERDETEKSI KUAT* jika ≥75% atau *INDIKASI GEJALA*), daftar gejala terpenuhi, saran penanganan, dan kandidat penyakit lain.	Jika berasal dari scan foto, tekan **Lengkapi Gejala Lain (Kuisioner)** untuk memverifikasi gejala non-visual (umur pohon, aroma busuk, atau pangkal batang).

1.6 Basis Pengetahuan Penyakit & Aturan Forward Chaining

Rule	Penyakit	Gejala Aturan	Indikator Visual di Foto Pelepah
`R1`	**P01** — Busuk Pangkal Batang (Ganoderma)	`G01, G02, G03, G04`	Klorosis / pelepah menguning luas (`G01`)
`R2`	**P02** — Busuk Pucuk (Bud Rot)	`G05, G06`	Area pembusukan coklat gelap basah (`G05`)
`R3`	**P03** — Penyakit Mahkota (Crown Disease)	`G07, G08, G09`	Verifikasi lengkap melalui kuisioner lapangan
`R4`	**P04** — Busuk Batang Atas (Upper Stem Rot)	`G10, G11`	Pelepah mengering coklat jerami luas (`G10`)

Rule	Penyakit	Gejala Aturan	Indikator Visual di Foto Pelepah
`R5`	**P05** — Antraknosa (Anthracnose)	`G12, G09`	Bercak gelap dikelilingi halo kuning di daun hijau (`G12`)
`R6`	**P06** — Bercak Daun Curvularia	`G13, G09`	Bintik kecil kuning-kecoklatan tersebar di daun hijau (`G13`)

Bagian II — Dokumentasi Kodingan Penting

2.1 Peta File Penting Proyek

Path File	Tanggung Jawab Utama
`lib/main.dart`	Entry point `PalmGuardApp`, konfigurasi tema gelap hijau sawit (`scaffoldBackgroundColor` & `ColorScheme.dark`).
`lib/utils/forward_chaining.dart`	Daftar 13 gejala (`G01` – `G13`), 6 penyakit (`P01` – `P06`), 6 rule (`R1` – `R6`), dan method `diagnose()`.
`lib/utils/palm_image_analyzer.dart`	Decode gambar via `dart:ui`, klasifikasi piksel HSV/ExG/kulit, validasi pelepah sawit, dan ekstraksi gejala visual.
`lib/pages/scan_page.dart`	Kontrol kamera (`camera`), tombol lampu kilat, pemilih foto galeri (`image_picker`), serta pipeline analisis gambar nyata.
`lib/pages/diagnosis_page.dart`	Kuisioner bertahap dengan dukungan pre-fill `initialSymptomIds` dan indikator jawaban sebelumnya.
`lib/pages/result_page.dart`	Menampilkan 3 kondisi hasil (bukan sawit, sawit sehat, atau indikasi penyakit), metrik citra, dan navigasi lanjutan.
`.github/workflows/flutter_ci.yml`	Workflow GitHub Actions untuk `flutter pub get`, `flutter analyze`, `flutter test`, dan build APK otomatis.
`test/palm_image_analyzer_test.dart`	Unit test validasi penolakan wajah/tembok/gelap/bidang polos serta deteksi daun sehat, menguning, dan halo bercak.

2.2 Engine Forward Chaining (`lib/utils/forward_chaining.dart`)

Setiap aturan dievaluasi terhadap daftar `selectedSymptomIds`. Tingkat keyakinan dihitung dari rasio gejala yang cocok dibagi total gejala pada aturan tersebut, lalu diurutkan dari confidence tertinggi.

lib/utils/forward_chaining.dart (baris 160–209)

```
  static const List<Rule> rules = [
    Rule(id: 'R1', symptomIds: ['G01', 'G02', 'G03', 'G04'], diseaseId: 'P01'),
    Rule(id: 'R2', symptomIds: ['G05', 'G06'], diseaseId: 'P02'),
    Rule(id: 'R3', symptomIds: ['G07', 'G08', 'G09'], diseaseId: 'P03'),
    Rule(id: 'R4', symptomIds: ['G10', 'G11'], diseaseId: 'P04'),
    Rule(id: 'R5', symptomIds: ['G12', 'G09'], diseaseId: 'P05'),
    Rule(id: 'R6', symptomIds: ['G13', 'G09'], diseaseId: 'P06'),
  ];

  /// Mendiagnosis penyakit kelapa sawit berdasarkan daftar symptomIds yang dipilih user.
  /// Menggunakan metode Forward Chaining.
  /// Mengembalikan list DiagnosisResult yang terurut berdasarkan confidence tertinggi.
  static List<DiagnosisResult> diagnose(List<String> selectedSymptomIds) {
    if (selectedSymptomIds.isEmpty) return [];

    List<DiagnosisResult> results = [];

    // Evaluasi setiap rule (Forward Chaining)
    for (var rule in rules) {
      List<String> matched = [];
      for (var sId in rule.symptomIds) {
```

```
        if (selectedSymptomIds.contains(sId)) {
          matched.add(sId);
        }
      }

      if (matched.isNotEmpty) {
        // Hitung persentase kecocokan (confidence)
        // confidence = (jumlah gejala yang cocok / total gejala pada rule) * 100
        double confidence = (matched.length / rule.symptomIds.length) * 100.0;

        // Dapatkan data Penyakit
        Disease disease = diseases.firstWhere((d) => d.id == rule.diseaseId);

        results.add(DiagnosisResult(
          disease: disease,
          confidence: confidence,
          matchedSymptoms: matched,
          totalSymptoms: rule.symptomIds.length,
        ));
      }
    }

    // Urutkan berdasarkan persentase kecocokan tertinggi
    results.sort((a, b) => b.confidence.compareTo(a.confidence));

    return results;
  }
}
```

2.3 Dekode & Klasifikasi Piksel Pelepah Sawit (`lib/utils/palm_image_analyzer.dart`)

Gambar terkompresi (JPEG/PNG) didekode secara native menggunakan `dart:ui` (tanpa dependensi pemrosesan gambar tambahan) pada lebar target 256 px agar cepat dan hemat memori, lalu setiap piksel diklasifikasikan menggunakan kombinasi ruang warna **HSV** dan indeks **Excess Green** (`ExG = 2G - R - B`).

lib/utils/palm_image_analyzer.dart — Dekode Gambar (baris 83–130)

```
class PalmImageAnalyzer {
  static const int _targetGridSize = 96;

  /// Mendekode byte gambar terkompresi (JPEG/PNG) menggunakan `dart:ui` lalu menganalisisnya.
  static Future<PalmImageAnalysis> analyzeImageBytes(Uint8List encodedBytes) async {
    if (encodedBytes.isEmpty) {
      return _rejected(
        error: PalmValidationError.invalidImage,
        title: 'Gambar Tidak Valid',
        detail: 'Data gambar kosong atau tidak dapat dibaca.',
      );
    }

    ui.Codec? codec;
    ui.FrameInfo? frame;
    try {
      // Decode pada resolusi yang efisien agar cepat dan hemat memori
      codec = await ui.instantiateImageCodec(
        encodedBytes,
        targetWidth: 256,
      );
      frame = await codec.getNextFrame();
      final byteData = await frame.image.toByteData(format: ui.ImageByteFormat.rawRgba);
      if (byteData == null) {
        return _rejected(
          error: PalmValidationError.invalidImage,
          title: 'Gagal Membaca Piksel',
          detail: 'Format piksel gambar tidak didukung.',
```

```
        );
      }

      return analyzeRgba(
        byteData.buffer.asUint8List(),
        frame.image.width,
        frame.image.height,
      );
    } catch (e) {
      return _rejected(
        error: PalmValidationError.invalidImage,
        title: 'Gagal Memproses Gambar',
        detail: 'Terjadi kesalahan saat membaca gambar: $e',
      );
    } finally {
      frame?.image.dispose();
      codec?.dispose();
    }
  }
```

lib/utils/palm_image_analyzer.dart — Klasifikasi Piksel (baris 557–618)

```
      brownDryRatio: dryBrownRatio,
      darkRotRatio: darkRotRatio,
      skinOrNonPlantRatio: combinedNonPlant,
      meanBrightness: meanBrightness,
      edgeDensity: edgeDensity,
    );
  }

  static _PixelClass _classifyPixel(
    int r,
    int g,
    int b,
    double rn,
    double gn,
    double bn,
    double h,
    double s,
    double v,
    double exG,
  ) {
    if (s < 0.15) {
      return _PixelClass.neutral;
    }

    // Kulit manusia / wajah: merah dominan, tanpa excess green, saturasi sedang
    final int maxRgb = math.max(r, math.max(g, b));
    final int minRgb = math.min(r, math.min(g, b));
    final bool isSkinRgb = r > 85 &&
        g > 35 &&
        b > 15 &&
        (maxRgb - minRgb) > 15 &&
        (r - g).abs() > 12 &&
        r > g &&
        r > b;
    final bool isSkinHsv = ((h >= 0.0 && h <= 42.0) || h >= 335.0) &&
        s >= 0.12 &&
        s <= 0.72 &&
        v >= 0.20 &&
        v <= 0.96;
    if (isSkinRgb && isSkinHsv && exG < 0.0) {
      return _PixelClass.skinTone;
    }

    // Daun hijau sawit (spektrum hijau dengan excess green positif)
```

```
    if (h >= 62.0 && h <= 165.0 && s >= 0.16 && v >= 0.12 && v <= 0.95 && exG > 0.03) {
      return _PixelClass.greenLeaf;
    }

    // Daun menguning / klorosis
    if (h >= 40.0 && h < 62.0 && s >= 0.22 && v >= 0.22 && gn >= rn * 0.72 && gn > bn * 1.25) {
      return _PixelClass.yellowLeaf;
    }

    // Pembusukan / bercak coklat gelap kehitaman
    if (h >= 8.0 && h < 45.0 && s >= 0.18 && v >= 0.08 && v < 0.34 && r > b) {
      return _PixelClass.darkRot;
    }

    // Daun mengering / nekrosis coklat jerami
    if (h >= 18.0 && h < 45.0 && s >= 0.15 && s <= 0.75 && v >= 0.34 && v <= 0.85) {
      return _PixelClass.dryBrown;
    }
```

2.4 Gerbang Validasi Pelepah Sawit & Ekstraksi Gejala Visual

Bagian ini mencegah objek non-sawit (seperti wajah/kulit di tengah bingkai, latar tembok netral, warna non-tanaman, atau bidang hijau polos tanpa tekstur) terdeteksi sebagai penyakit sawit. Jika lolos validasi, rasio area kuning, busuk gelap, coklat kering, bercak halo, dan bintik tersebar dipetakan ke kode gejala `G01`, `G05`, `G10`, `G12`, dan `G13`.

lib/utils/palm_image_analyzer.dart — Validasi Non-Sawit (baris 328–432)

```
    // Tolak wajah / kulit manusia (terutama jika berada di tengah frame kamera)
    if ((centerSkinRatio >= 0.18 && centerGreenRatio < 0.15) ||
        ((centerSkinRatio >= 0.18 || skinRatio >= 0.25) && greenRatio < 0.18)) {
      return PalmImageAnalysis(
        isPalmFrond: false,
        validationError: PalmValidationError.nonPlantSkinOrFace,
        statusTitle: 'Bukan Pelepah Sawit',
        statusDetail:
            'Terdeteksi objek non-tanaman (seperti wajah/kulit atau objek berwarna krem/oranye). Fokuskan kamera
hanya pada pelepah atau helai daun kelapa sawit.',
        detectedSymptomIds: const [],
        greenLeafRatio: greenRatio,
        yellowChlorosisRatio: yellowRatio,
        brownDryRatio: dryBrownRatio,
        darkRotRatio: darkRotRatio,
        skinOrNonPlantRatio: combinedNonPlant,
        meanBrightness: meanBrightness,
        edgeDensity: edgeDensity,
      );
    }

    // Tolak warna non-vegetasi dominan (langit biru, baju merah/ungu, dll.)
    if ((nonPlantColorRatio >= 0.35 || centerNonPlantRatio >= 0.35) && greenRatio < 0.20) {
      return PalmImageAnalysis(
        isPalmFrond: false,
        validationError: PalmValidationError.nonPlantColor,
        statusTitle: 'Bukan Pelepah Sawit',
        statusDetail:
            'Warna dominan pada foto tidak sesuai dengan karakteristik vegetasi pelepah atau daun kelapa sawit.',
        detectedSymptomIds: const [],
        greenLeafRatio: greenRatio,
        yellowChlorosisRatio: yellowRatio,
        brownDryRatio: dryBrownRatio,
        darkRotRatio: darkRotRatio,
        skinOrNonPlantRatio: combinedNonPlant,
        meanBrightness: meanBrightness,
        edgeDensity: edgeDensity,
      );
    }

    // Tolak bidang netral/polos (tembok, meja abu-abu/putih, latar belakang ruangan)
    if (neutralRatio >= 0.72 && greenRatio < 0.12) {
      return PalmImageAnalysis(
        isPalmFrond: false,
        validationError: PalmValidationError.neutralBackground,
        statusTitle: 'Bukan Pelepah Sawit',
        statusDetail:
            'Gambar didominasi bidang netral atau polos (tembok, lantai, atau ruangan). Arahkan kamera lebih dekat
ke pelepah sawit.',
        detectedSymptomIds: const [],
        greenLeafRatio: greenRatio,
        yellowChlorosisRatio: yellowRatio,
        brownDryRatio: dryBrownRatio,
        darkRotRatio: darkRotRatio,
        skinOrNonPlantRatio: combinedNonPlant,
```

```
        meanBrightness: meanBrightness,
        edgeDensity: edgeDensity,
      );
    }

    // Wajib memiliki komponen vegetasi hijau/daun sawit yang memadai di keseluruhan dan tengah frame
    final double totalFoliage = greenRatio + yellowRatio;
    final double centerFoliage = centerGreenRatio + centerYellowRatio;
    final bool hasEnoughPalmFoliage =
        (greenRatio >= 0.10 || (totalFoliage >= 0.28 && greenRatio >= 0.05 && skinRatio < 0.15)) &&
            centerFoliage >= 0.10;

    if (!hasEnoughPalmFoliage) {
      return PalmImageAnalysis(
        isPalmFrond: false,
        validationError: PalmValidationError.noPalmFoliage,
        statusTitle: 'Pelepah Sawit Tidak Terdeteksi',
        statusDetail:
            'Objek di dalam area fokus tidak menunjukkan karakteristik daun/pelepah kelapa sawit. Pastikan pelepah
berada di tengah bingkai kamera.',
        detectedSymptomIds: const [],
        greenLeafRatio: greenRatio,
        yellowChlorosisRatio: yellowRatio,
        brownDryRatio: dryBrownRatio,
        darkRotRatio: darkRotRatio,
        skinOrNonPlantRatio: combinedNonPlant,
        meanBrightness: meanBrightness,
        edgeDensity: edgeDensity,
      );
    }

    // Tolak bidang hijau polos seragam tanpa tekstur daun/lidi (misal tembok cat hijau/layar hijau)
    if (edgeDensity < 0.015 &&
        greenExGStdDev < 0.035 &&
        yellowRatio < 0.02 &&
        darkRotRatio < 0.01) {
      return PalmImageAnalysis(
        isPalmFrond: false,
        validationError: PalmValidationError.flatGreenSurface,
        statusTitle: 'Bukan Pelepah Sawit',
        statusDetail:
            'Permukaan hijau terlihat polos dan seragam tanpa tekstur helai daun atau lidi pelepah sawit.',
        detectedSymptomIds: const [],
        greenLeafRatio: greenRatio,
        yellowChlorosisRatio: yellowRatio,
        brownDryRatio: dryBrownRatio,
        darkRotRatio: darkRotRatio,
        skinOrNonPlantRatio: combinedNonPlant,
        meanBrightness: meanBrightness,
        edgeDensity: edgeDensity,
      );
    }
```

lib/utils/palm_image_analyzer.dart — Pemetaan Gejala Visual (baris 484–532)

```
          }
        }
      }
    }

    final double haloSpotRatio = plantPixels > 0 ? haloSpotPixels / plantPixels : 0.0;
    final double scatteredSpotRatio = plantPixels > 0 ? scatteredSpotPixels / plantPixels : 0.0;

    final List<String> detectedSymptoms = [];

    final bool isClearlyHealthy = relGreen >= 0.78 &&
```

```
        relYellow < 0.15 &&
        relDryBrown < 0.08 &&
        relDarkRot < 0.05 &&
        haloSpotRatio < 0.010 &&
        scatteredSpotRatio < 0.03;

    if (!isClearlyHealthy) {
      // G12: Bercak coklat kehitaman dikelilingi warna kuning (halo) -> Antraknosa (P05)
      if (haloSpotRatio >= 0.010 && relGreen >= 0.25) {
        detectedSymptoms.add('G12');
      }

      // G13: Bercak kecil bulat kuning kecoklatan tersebar -> Curvularia (P06)
      if (scatteredSpotRatio >= 0.03 && !detectedSymptoms.contains('G12')) {
        detectedSymptoms.add('G13');
      }

      // G05: Pembusukan basah berwarna coklat gelap -> Busuk Pucuk (P02)
      if (relDarkRot >= 0.16) {
        detectedSymptoms.add('G05');
      }

      // G01: Pelepah daun menguning luas (klorosis berat) -> indikasi visual Ganoderma (P01)
      if (relYellow >= 0.22) {
        detectedSymptoms.add('G01');
      }

      // G10: Pelepah mengering/coklat jerami luas -> indikasi Busuk Batang Atas (P04)
      if (relDryBrown >= 0.20) {
        detectedSymptoms.add('G10');
      }

      // Jika ada gangguan visual kombinasi ringan-sedang yang belum melewati threshold tunggal
      if (detectedSymptoms.isEmpty) {
        if (relDarkRot >= 0.08) {
          detectedSymptoms.add('G05');
        } else if (relYellow >= relDryBrown && relYellow >= 0.12) {
          detectedSymptoms.add('G01');
```

2.5 Pipeline Scan Kamera & Galeri (`lib/pages/scan_page.dart`)

Baik foto dari kamera langsung (`_takePictureAndAnalyze`) maupun foto yang sudah ada dari galeri (`_pickFromGalleryAndAnalyze`) masuk ke fungsi yang sama yaitu `_runVisualAnalysisPipeline` . Mesin `ForwardChainingEngine.diagnose` hanya dijalankan apabila `analysis.isPalmFrond == true` .

lib/pages/scan_page.dart (baris 156–293)

```
  Future<void> _takePictureAndAnalyze() async {
    if (_isScanning) return;

    final controller = _cameraController;
    if (controller == null || !controller.value.isInitialized) {
      ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(
          content: Text(
            'Kamera belum aktif. Pastikan izin kamera diberikan lalu coba lagi.',
          ),
        ),
      );
      return;
    }

    try {
      setState(() {
        _isScanning = true;
```

```
        _scanningStatus = 'Mengambil gambar pelepah...';
        _scanningProgress = 0.1;
      });

      final XFile image = await controller.takePicture();
      final Uint8List bytes = await image.readAsBytes();

      if (_isFlashOn) {
        try {
          await controller.setFlashMode(FlashMode.off);
        } catch (_) {}
      }

      if (!mounted) return;
      await _runVisualAnalysisPipeline(bytes);
    } catch (e) {
      if (!mounted) return;
      setState(() {
        _isScanning = false;
      });
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text('Gagal mengambil gambar: $e')),
      );
    }
  }

  Future<void> _pickFromGalleryAndAnalyze() async {
    if (_isScanning) return;

    try {
      final XFile? picked = await _imagePicker.pickImage(
        source: ImageSource.gallery,
        imageQuality: 90,
      );
      if (picked == null || !mounted) return;

      final Uint8List bytes = await picked.readAsBytes();
      if (!mounted) return;

      if (_isFlashOn && _cameraController != null) {
        try {
          await _cameraController!.setFlashMode(FlashMode.off);
        } catch (_) {}
      }

      setState(() {
        _isScanning = true;
        _scanningStatus = 'Membaca gambar dari galeri...';
        _scanningProgress = 0.1;
      });

      await _runVisualAnalysisPipeline(bytes);
    } catch (e) {
      if (!mounted) return;
      setState(() {
        _isScanning = false;
      });
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text('Gagal memilih gambar dari galeri: $e')),
      );
    }
  }

  Future<void> _runVisualAnalysisPipeline(Uint8List imageBytes) async {
    setState(() {
      _capturedImageBytes = imageBytes;
      _isFlashOn = false;
```

```
      _scanningProgress = 0.2;
      _scanningStatus = 'Memvalidasi objek pelepah sawit...';
    });

    // Jalankan analisis citra nyata secara paralel dengan progres UI singkat
    final analysisFuture = PalmImageAnalyzer.analyzeImageBytes(imageBytes);

    final steps = <(double, String)>[
      (0.4, 'Mengecek spektrum vegetasi & tekstur daun...'),
      (0.65, 'Menganalisis klorosis kuning & bercak daun...'),
      (0.85, 'Mencocokkan gejala visual ke Forward Chaining...'),
    ];

    for (final (progress, status) in steps) {
      await Future<void>.delayed(const Duration(milliseconds: 350));
      if (!mounted) return;
      setState(() {
        _scanningProgress = progress;
        _scanningStatus = status;
      });
    }

    final PalmImageAnalysis analysis = await analysisFuture;
    if (!mounted) return;

    setState(() {
      _scanningProgress = 1.0;
      _scanningStatus = analysis.isPalmFrond
          ? 'Pelepah sawit terverifikasi. Menyiapkan hasil...'
          : 'Objek bukan pelepah sawit. Menyiapkan laporan...';
    });

    await Future<void>.delayed(const Duration(milliseconds: 250));
    if (!mounted) return;

    // Hanya jalankan Forward Chaining jika gambar benar-benar terdeteksi sebagai pelepah sawit
    final List<String> detectedSymptoms =
        analysis.isPalmFrond ? analysis.detectedSymptomIds : const [];
    final List<DiagnosisResult> diagnosisResults = analysis.isPalmFrond
        ? ForwardChainingEngine.diagnose(detectedSymptoms)
        : const [];

    Navigator.pushReplacement(
      context,
      MaterialPageRoute(
        builder: (context) => ResultPage(
          selectedSymptomIds: detectedSymptoms,
          results: diagnosisResults,
          imageBytes: imageBytes,
          visualAnalysis: analysis,
        ),
      ),
    );
  }
```

2.6 Penanganan 3 Kondisi di Halaman Hasil (`lib/pages/result_page.dart`)

Halaman hasil membedakan status penolakan objek non-sawit (`_isNotPalmFrond`), kondisi sehat, dan kondisi bergejala, serta menyediakan tombol lanjutan ke `DiagnosisPage(initialSymptomIds: selectedSymptomIds)` .

lib/pages/result_page.dart (baris 23–142)

```
  bool get _isNotPalmFrond =>
      visualAnalysis != null && !visualAnalysis!.isPalmFrond;

  @override
  Widget build(BuildContext context) {
    final primaryResult = results.isNotEmpty ? results.first : null;
    final otherResults =
        results.length > 1 ? results.sublist(1) : <DiagnosisResult>[];

    return Scaffold(
      backgroundColor: const Color(0xFF0C1910),
      appBar: AppBar(
        backgroundColor: const Color(0xFF0C1910),
        elevation: 0,
        leading: IconButton(
          icon: const Icon(Icons.close, color: Colors.white),
          onPressed: () {
            Navigator.popUntil(context, (route) => route.isFirst);
          },
        ),
        title: Text(
          _isNotPalmFrond ? 'Validasi Objek Scan' : 'Hasil Diagnosis',
          style: const TextStyle(
            color: Colors.white,
            fontSize: 18,
            fontWeight: FontWeight.bold,
          ),
        ),
        centerTitle: true,
      ),
      body: SafeArea(
        child: SingleChildScrollView(
          child: Padding(
            padding: const EdgeInsets.symmetric(horizontal: 24.0, vertical: 16.0),
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                if (imageBytes != null || imagePath != null) ...[
                  _buildCapturedImagePreview(),
                  const SizedBox(height: 20),
                ],

                if (_isNotPalmFrond) ...[
                  // Kondisi 1: Objek bukan pelepah/daun sawit -> Jangan deteksi penyakit sawit
                  _buildNotPalmFrondCard(context, visualAnalysis!),
                  const SizedBox(height: 20),
                  _buildScanTipsCard(),
                  const SizedBox(height: 24),
                ] else if (primaryResult == null) ...[
                  // Kondisi 2: Pelepah sawit sehat / tidak ada gejala terdeteksi
                  _buildNoDiseaseCard(context),
                  if (visualAnalysis != null) ...[
                    const SizedBox(height: 20),
                    _buildVisualMetricsSection(visualAnalysis!),
                  ],
                  const SizedBox(height: 24),
                ] else ...[
                  // Kondisi 3: Penyakit sawit terdeteksi
```

```
                _buildPrimaryDiagnosisCard(context, primaryResult),
                const SizedBox(height: 20),

                if (visualAnalysis != null) ...[
                  _buildVisualMetricsSection(visualAnalysis!),
                  const SizedBox(height: 20),
                ],

                _buildMatchedSymptomsSection(primaryResult),
                const SizedBox(height: 20),

                _buildSolutionSection(primaryResult),
                const SizedBox(height: 24),

                if (otherResults.isNotEmpty) ...[
                  const Text(
                    'Kemungkinan Penyakit Lain',
                    style: TextStyle(
                      color: Colors.white,
                      fontSize: 16,
                      fontWeight: FontWeight.bold,
                    ),
                  ),
                  const SizedBox(height: 12),
                  ...otherResults.map(_buildAlternativeDiagnosisItem),
                  const SizedBox(height: 24),
                ],
              ],

              // Tombol Lengkapi Gejala via Kuisioner (jika datang dari scan kamera sawit)
              if (!_isNotPalmFrond && visualAnalysis != null) ...[
                OutlinedButton.icon(
                  onPressed: () {
                    Navigator.pushReplacement(
                      context,
                      MaterialPageRoute(
                        builder: (context) => DiagnosisPage(
                          initialSymptomIds: selectedSymptomIds,
                        ),
                      ),
                    );
                  },
                  icon: const Icon(Icons.assignment_outlined, color: Color(0xFF10B981)),
                  label: const Text(
                    'Lengkapi Gejala Lain (Kuisioner)',
                    style: TextStyle(
                      color: Colors.white,
                      fontSize: 14,
                      fontWeight: FontWeight.w600,
                    ),
                  ),
                  style: OutlinedButton.styleFrom(
                    side: const BorderSide(color: Color(0xFF10B981), width: 1.5),
                    minimumSize: const Size(double.infinity, 50),
                    shape: RoundedRectangleBorder(
                      borderRadius: BorderRadius.circular(12),
                    ),
                  ),
                ),
                const SizedBox(height: 12),
```

2.7 Pipeline CI/CD & Build APK Otomatis (`.github/workflows/flutter_ci.yml`)

Setiap push ke branch `main` atau `master` otomatis menjalankan analisis statis, seluruh unit/widget test, serta membangun file APK rilis Android yang siap diunduh.

.github/workflows/flutter_ci.yml

```
name: Flutter CI & Build APK

on:
  push:
    branches:
      - main
      - master
  pull_request:
    branches:
      - main
      - master
  workflow_dispatch:

jobs:
  analyze_test_and_build:
    name: Analyze, Test & Build Android APK
    runs-on: ubuntu-latest

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Setup Java 17
        uses: actions/setup-java@v4
        with:
          distribution: 'temurin'
          java-version: '17'

      - name: Setup Flutter (stable)
        uses: subosito/flutter-action@v2
        with:
          channel: 'stable'
          cache: true

      - name: Install dependencies
        run: flutter pub get

      - name: Run static analysis
        run: |
          set +e
          flutter analyze > analyze.log 2>&1
          EXIT_CODE=$?
          cat analyze.log
          if [ $EXIT_CODE -ne 0 ]; then
            while IFS= read -r line; do
              [ -n "$line" ] && echo "::error::$line"
            done < analyze.log
          fi
          exit $EXIT_CODE

      - name: Run unit & widget tests
        run: |
          set +e
          flutter test -r expanded > test.log 2>&1
          EXIT_CODE=$?
          cat test.log
          if [ $EXIT_CODE -ne 0 ]; then
            grep -A 2 -E "(The following assertion was thrown:|The following message was thrown:|\[E\])" test.log |
grep -v -E "^(--|The following assertion was thrown:|The following message was thrown:)$" | head -n 10 | while IFS=
read -r line; do
              echo "::error::$line"
            done
          fi
          exit $EXIT_CODE
```

```
      - name: Build Android Release APK
        run: flutter build apk --release

      - name: Upload APK artifact
        uses: actions/upload-artifact@v4
        with:
          name: sawit-detector-release-apk
          path: build/app/outputs/flutter-apk/app-release.apk
          retention-days: 14
```